MENJALANKAN CRUD API dengan Golang, MySQL dan Redis

LAPORAN TOPIK KHUSUS



SEMESTER VI

DISUSUN OLEH

FARHAN RAUF JUFADLI

2211082009

DOSEN PENGAMPU

Dr. Yulherniwati, S.Kom., M.T

TEKNOLOGI REKAYASA PERANGKAT LUNAK

TEKNOLOGI INFORMASI

POLITEKNIK NEGERI PADANG

2024 / 2025

A. LANDASAN TEORI

Dalam dunia pengembangan aplikasi berbasis web, optimalisasi performa merupakan aspek penting yang menentukan pengalaman pengguna serta efisiensi sistem. Salah satu tantangan terbesar dalam pengelolaan sistem berbasis data adalah bagaimana menangani permintaan data yang besar tanpa mengorbankan kecepatan dan efisiensi sumber daya.

Salah satu teknik yang umum digunakan untuk mengatasi masalah ini adalah penggunaan caching sebagai lapisan perantara antara aplikasi dan database utama. Caching bertujuan untuk mengurangi beban kerja pada database dan meningkatkan kecepatan akses data dengan menyimpan hasil query yang sering digunakan di dalam memori.

Dalam penelitian ini, kita akan membahas integrasi MySQL sebagai database utama dengan Redis sebagai caching layer untuk meningkatkan performa aplikasi web.

a) Pengelolaan Data dalam Aplikasi Web

Dalam sistem berbasis web, data menjadi elemen inti yang terus dikelola, diperbarui, dan diakses oleh pengguna. Sistem pengelolaan data dalam aplikasi web terdiri dari dua bagian utama:

1. Sistem Manajemen Basis Data (DBMS) – Bertanggung jawab untuk menyimpan, mengorganisir, dan mengakses data dengan cara yang efisien. DBMS dapat berupa:
    - Relational Database Management System (RDBMS) seperti MySQL, PostgreSQL, dan SQL Server, yang menggunakan model tabel dengan relasi antar data.
    - NoSQL Database seperti MongoDB, Cassandra, dan Redis, yang tidak menggunakan model tabel, melainkan berbasis dokumen atau key-value store.
2. Caching Layer – Digunakan untuk menyimpan data yang sering diakses di dalam memori, sehingga mengurangi jumlah permintaan langsung ke database dan meningkatkan performa aplikasi.

Salah satu tantangan utama dalam sistem berbasis database adalah bagaimana menangani permintaan data yang terus meningkat tanpa menyebabkan bottleneck pada server. Ketika jumlah pengguna bertambah, sistem perlu menangani banyak permintaan baca dan tulis ke database secara bersamaan. Jika semua permintaan langsung diarahkan ke database utama, hal ini dapat menyebabkan lonjakan beban kerja dan memperlambat waktu respons aplikasi.

b) MySQL sebagai Database Utama

MySQL adalah sistem manajemen basis data relasional (RDBMS) yang berbasis SQL (Structured Query Language) untuk menyimpan dan mengelola data. MySQL telah menjadi pilihan utama dalam pengembangan aplikasi web karena berbagai keunggulannya, antara lain:

- Mendukung transaksi ACID (Atomicity, Consistency, Isolation, Durability) yang menjamin keakuratan dan keandalan data.

- Menyediakan indeksasi dan optimasi query untuk meningkatkan efisiensi pencarian data.
- Memiliki skalabilitas tinggi, dapat digunakan dalam berbagai ukuran aplikasi dari yang sederhana hingga enterprise.

Namun, meskipun MySQL sangat andal dalam menangani transaksi, ia memiliki beberapa keterbatasan terutama dalam hal waktu eksekusi query dan skalabilitas. Saat jumlah permintaan data meningkat, sistem dapat mengalami bottleneck akibat beberapa faktor:

1. Overhead koneksi ke database – Setiap permintaan ke MySQL membutuhkan koneksi baru atau pemakaian kembali koneksi yang ada, yang dapat menyebabkan latensi tambahan.
2. Eksekusi query yang kompleks – Query yang melibatkan JOIN antar tabel atau perhitungan agregat bisa memerlukan waktu lebih lama untuk dieksekusi.
3. I/O Disk yang tinggi – MySQL menyimpan data di disk, sehingga setiap permintaan baca/tulis memerlukan akses ke penyimpanan fisik, yang lebih lambat dibandingkan memori.

Untuk mengatasi masalah ini, caching menjadi solusi utama dengan menyimpan data yang sering diakses di dalam memori agar bisa digunakan kembali tanpa harus mengakses MySQL setiap kali ada permintaan data.

c) Redis sebagai Sistem Caching

Redis (Remote Dictionary Server) adalah sistem penyimpanan data berbasis key-value yang berjalan di dalam memori (RAM). Redis dirancang untuk menyediakan akses data yang sangat cepat dengan latensi sangat rendah, membuatnya ideal digunakan sebagai sistem caching dalam aplikasi web.

Keunggulan Redis dibandingkan Database Konvensional:

1. Kecepatan tinggi – Karena berjalan di RAM, Redis dapat memproses jutaan permintaan per detik, jauh lebih cepat dibandingkan database berbasis disk seperti MySQL.
2. Struktur data fleksibel – Redis mendukung berbagai tipe data seperti String, Hash, List, Set, Sorted Set, yang membuatnya lebih fleksibel untuk berbagai skenario caching.
3. Mekanisme Persistence – Meskipun berjalan di memori, Redis juga menyediakan fitur snapshotting dan append-only file (AOF) untuk menyimpan data ke disk dan mencegah kehilangan data saat sistem restart.
4. Mendukung TTL (Time-To-Live) – Redis memungkinkan penghapusan otomatis data dalam cache setelah waktu tertentu, sehingga data tidak terus menerus memenuhi memori.

d) Arsitektur Integrasi MySQL dan Redis

Integrasi Redis dan MySQL dilakukan menggunakan pendekatan cache-aside. Pendekatan ini bekerja dengan cara:

1. Aplikasi pertama-tama mengecek Redis untuk mencari data yang diminta (Cache Hit).
    - Jika data tersedia, aplikasi langsung mengambilnya dari Redis tanpa perlu mengakses MySQL.
2. Jika data tidak ditemukan di Redis (Cache Miss), aplikasi akan mengambil data dari MySQL.
    - Setelah mendapatkan data dari MySQL, aplikasi akan menyimpan data tersebut di Redis agar bisa digunakan pada permintaan berikutnya.
3. Ketika data di MySQL diperbarui, Redis harus diperbarui atau data lama harus dihapus.

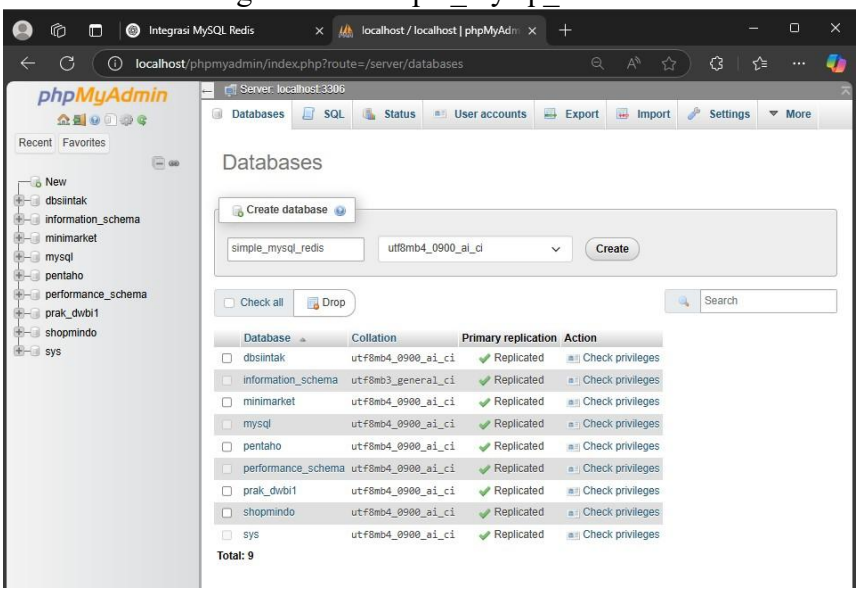
Pendekatan ini memberikan keseimbangan antara kecepatan akses data dan konsistensi informasi dalam sistem.

B. ALAT

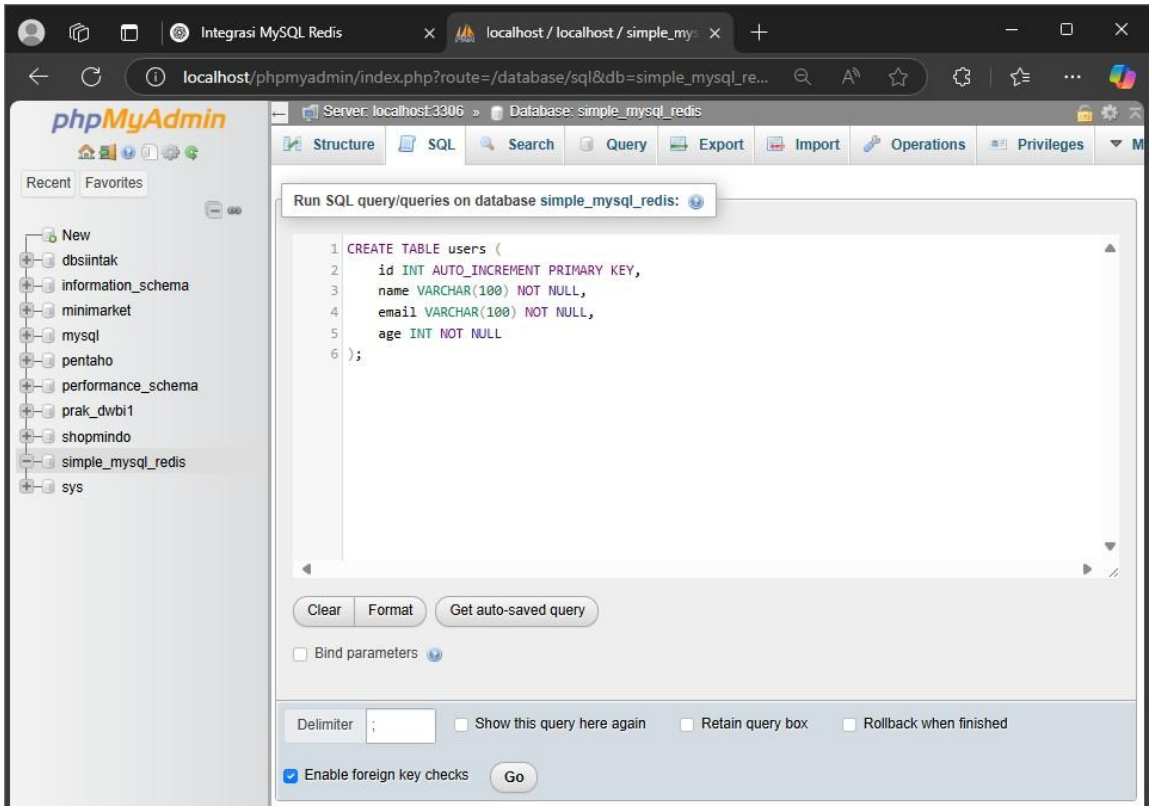
1) Visual Studio Code++
2) Command Prompt
3) Golang
4) MySQL (Laragon/XAMPP)
5) Redis
6) Postman

C. LANGKAH KERJA

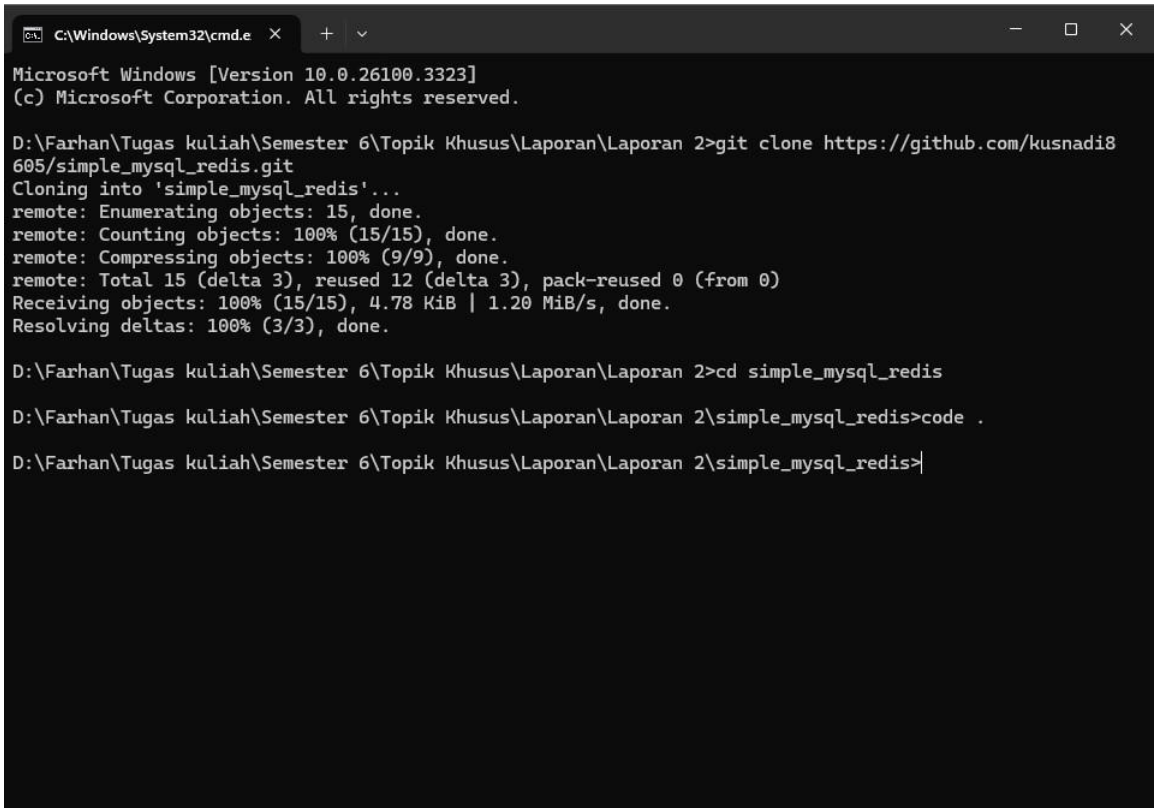
1) Buat database dengan nama simple_mysql_redis.



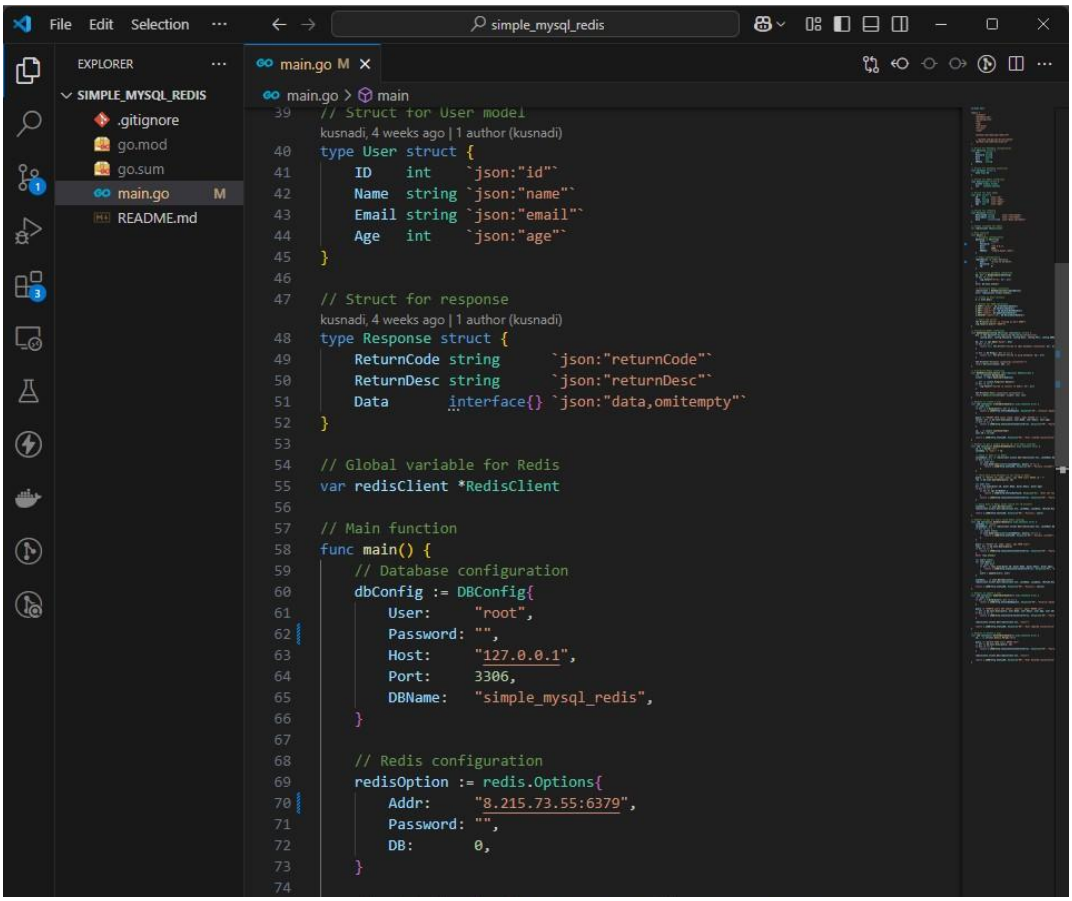
2) Buat tabel user.



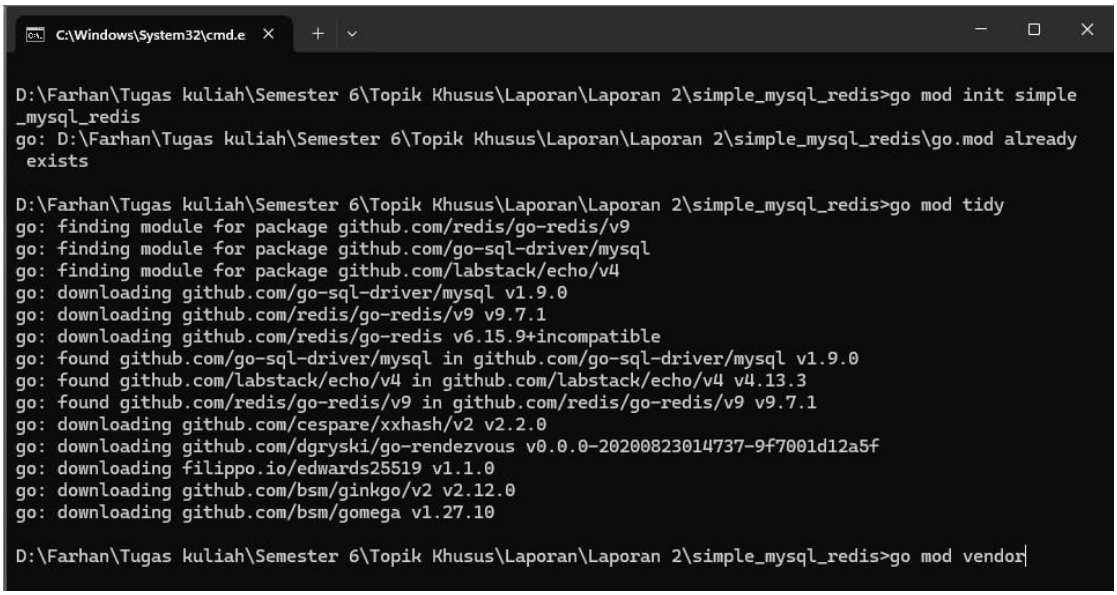
3) Lakukan clone repository dari github.



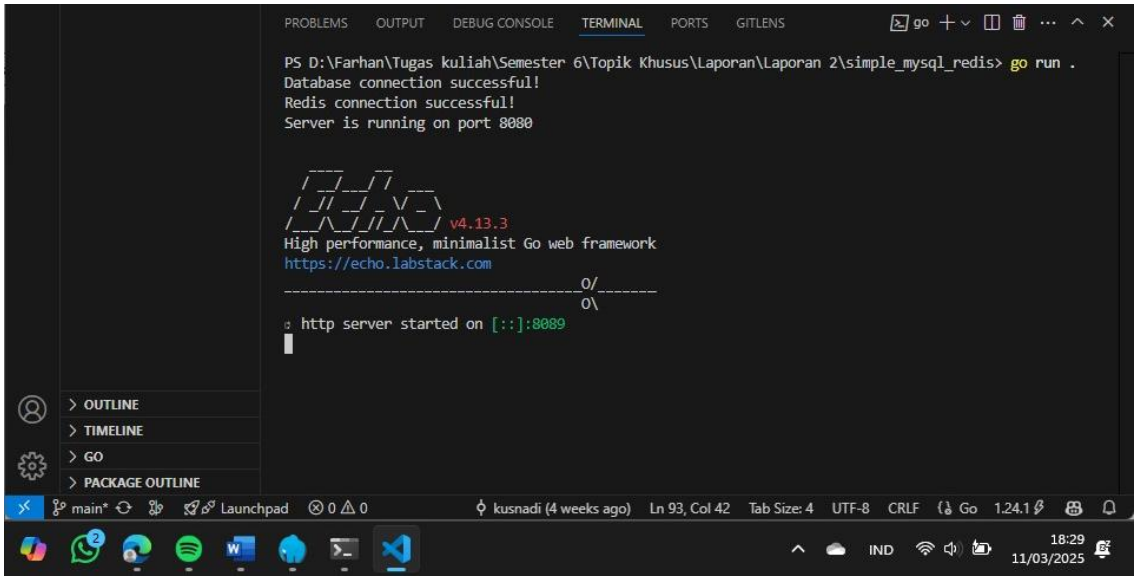
4) Konfigurasi database dan redis agar bisa saling terhubung pada file main.go. Pada dbConfig hapus atau kosongkan password. Pada konfigurasi Redis gunakan IP Onlinenya yaitu 8.215.73.55 menggantikan localhost.



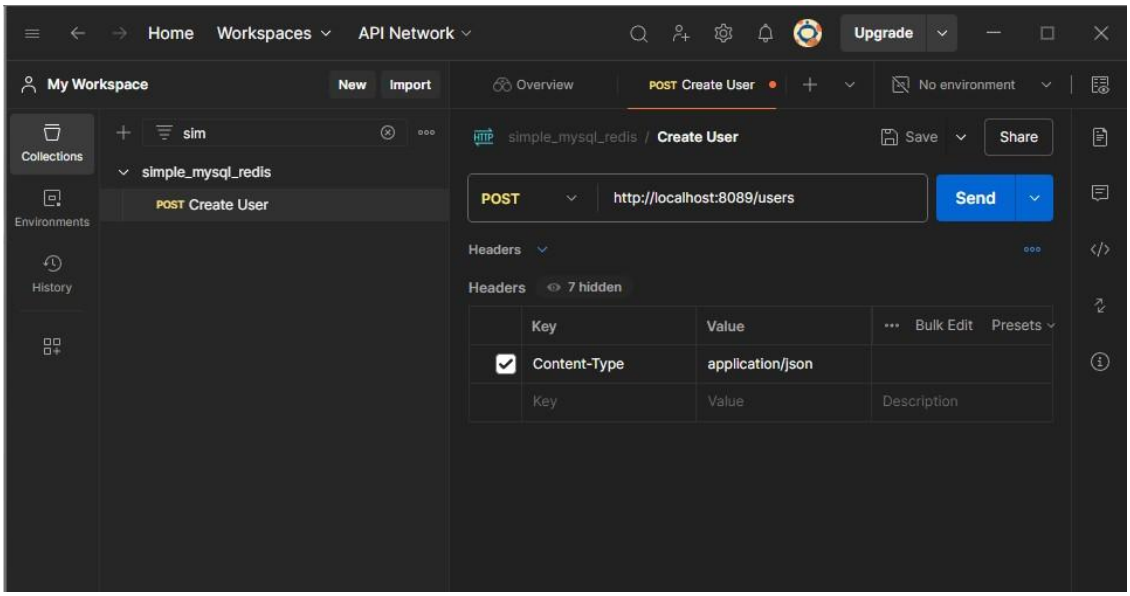
5) Instal dependensi yang di perlukan.



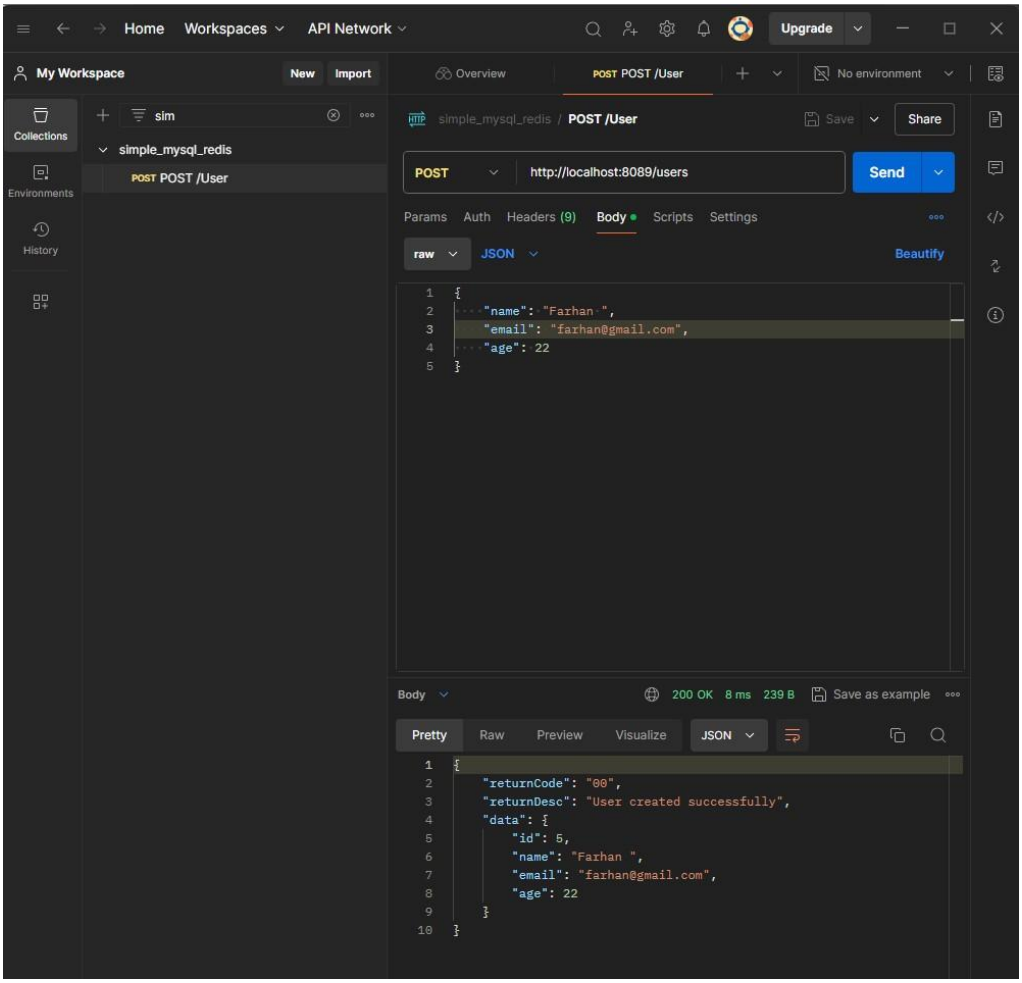
6) Jalankan server golang



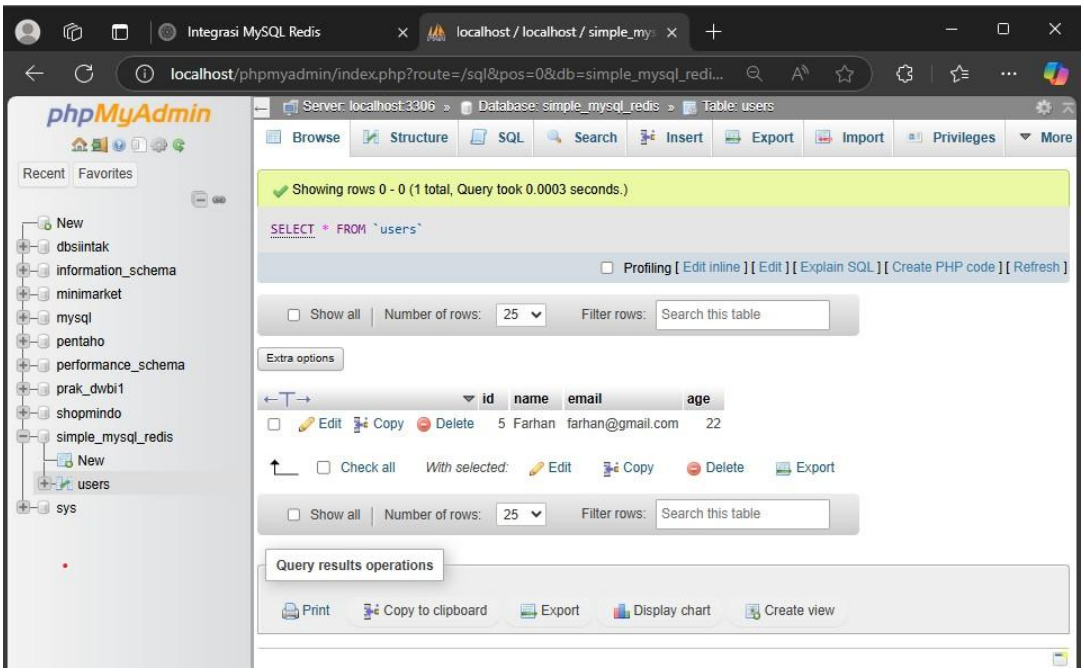
7) Selanjutnya menjalankan CRUD API dengan Postman.
8) Pertama **Create User,** lakukan **POST** pada postman dengan url http://localhost:8089/users. Pada **Header** isi **Key** dengan **Content-Type** dan **Values** dengan **application/json**



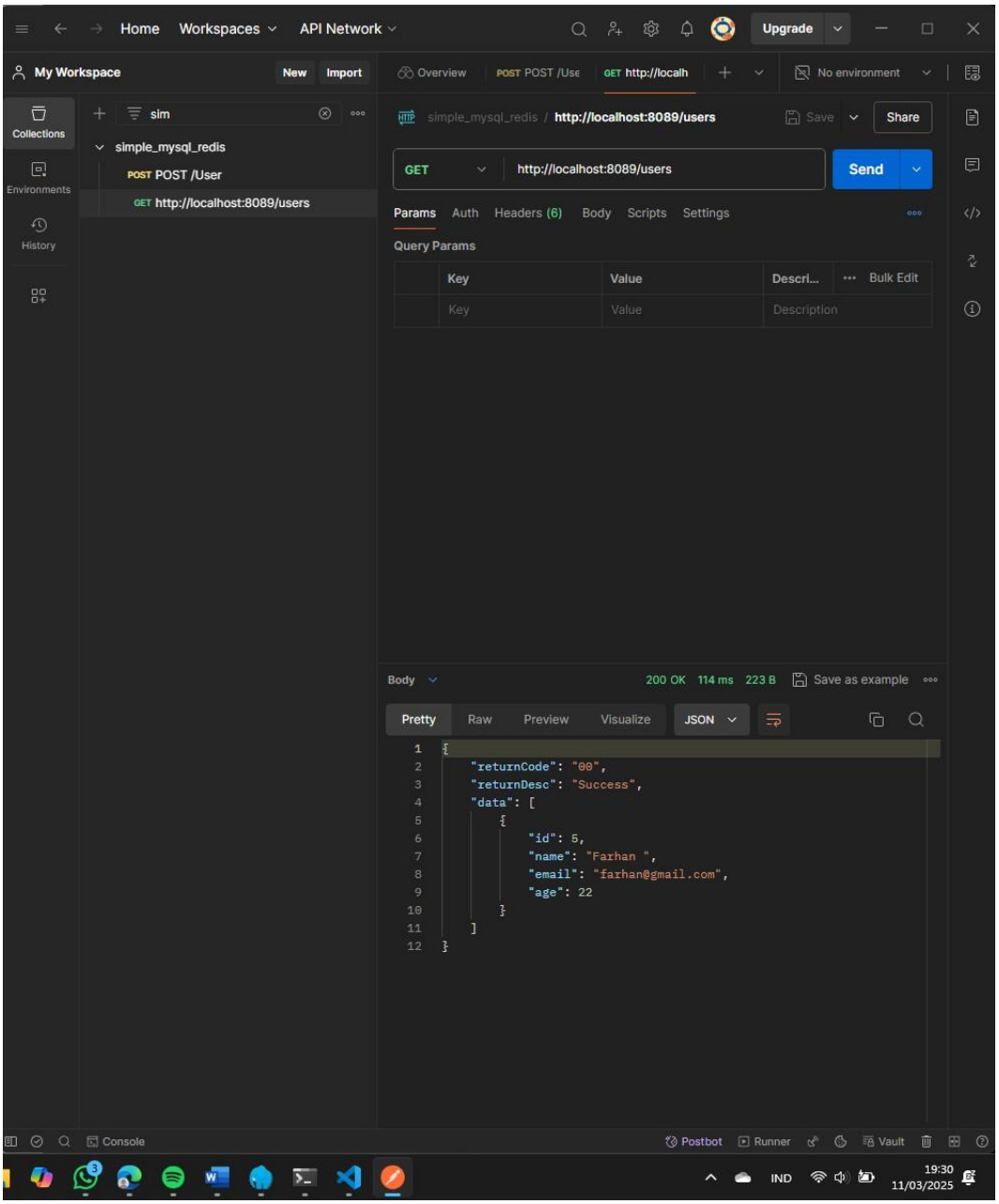
Selanjutnya pada bagian body pilih raw dan ketikan json yang ingin diisikan baik nama, email, age lalu lakukan send untuk lakukan POST API. Jika berhasil maka akan muncul response JSON dari server.



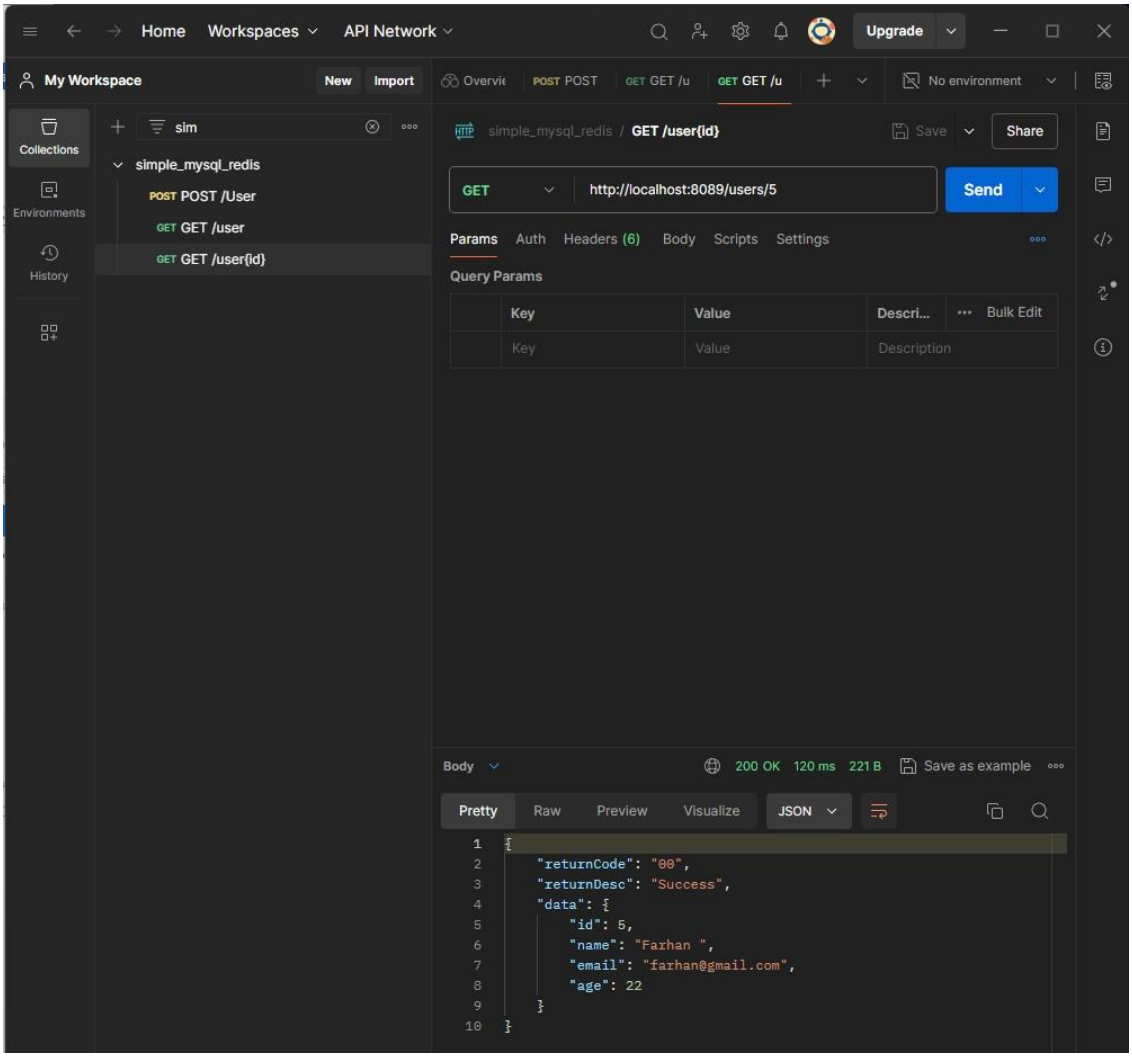
Data berhasil masuk ke tabel di database localhost.



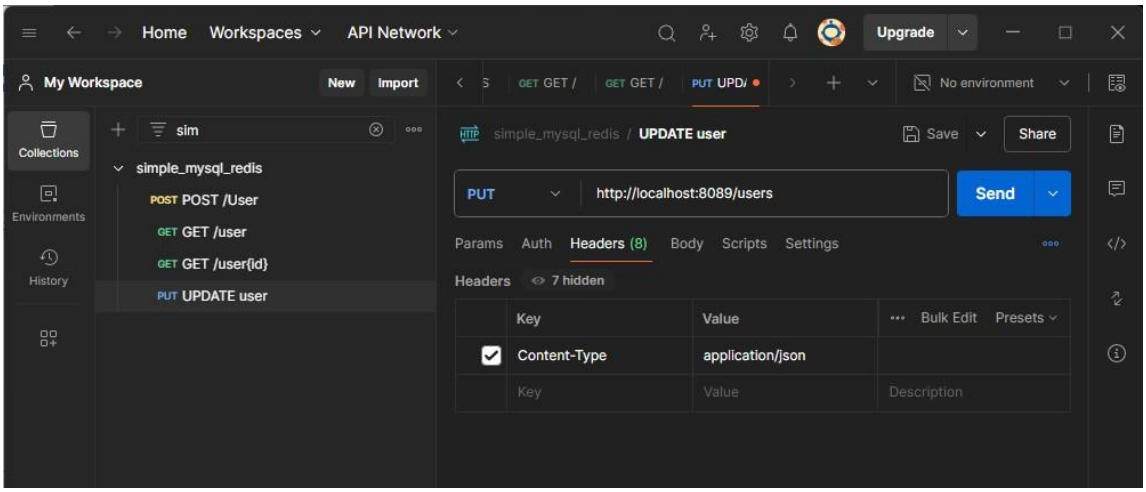
9) Kedua **Get All Users,** lakukan **GET** pada Postman dengan url http://localhost:8089/users untuk **GET** semua data yang ada. Jika berhasil maka muncul response.



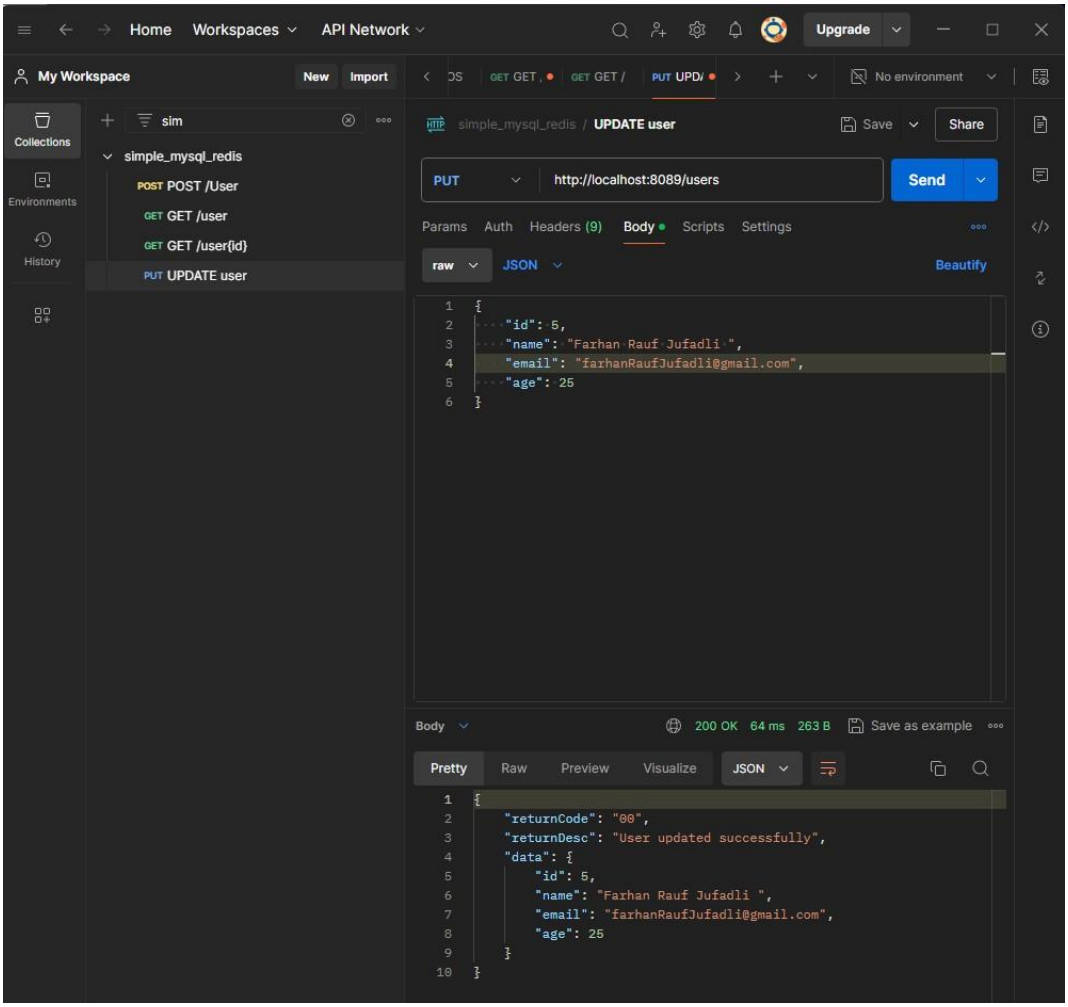
10) Ketiga **Get Single User,** lakukan **GET** pada Postman dengan url http://localhost:8089/users/5 lalu Send untuk **GET** salah satu data berdasarkan id. Jika berhasil maka akan muncul response.



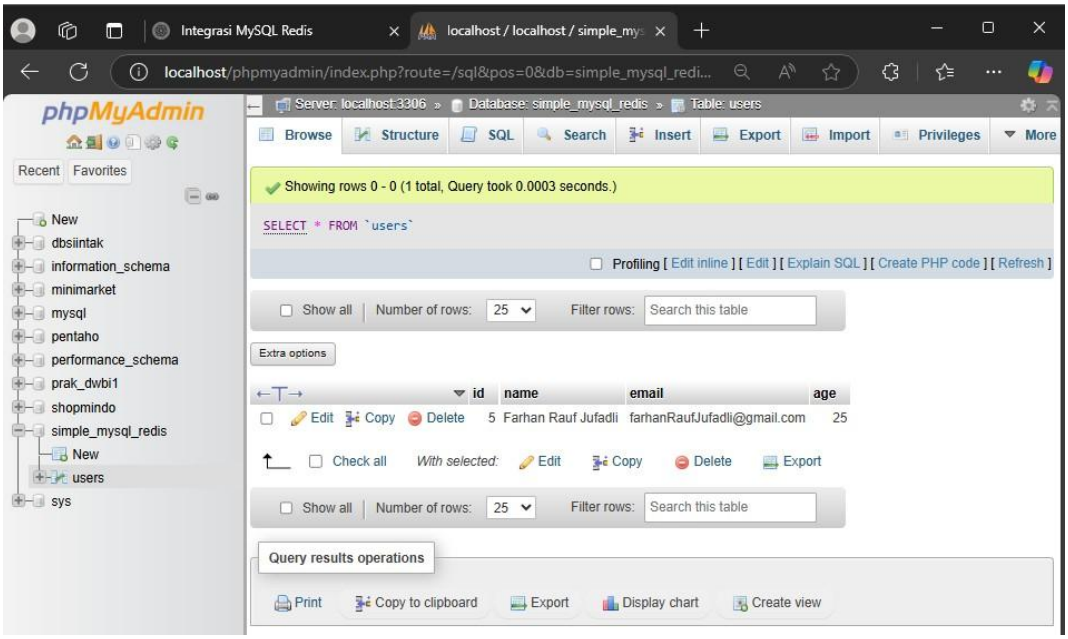
11) Keempat **Update User**, lakukan PUT pada Postman dengan url http://localhost:8089/users. Pada **Header** isi **Key** dengan **Content-Type** dan **Values** dengan **application/json**



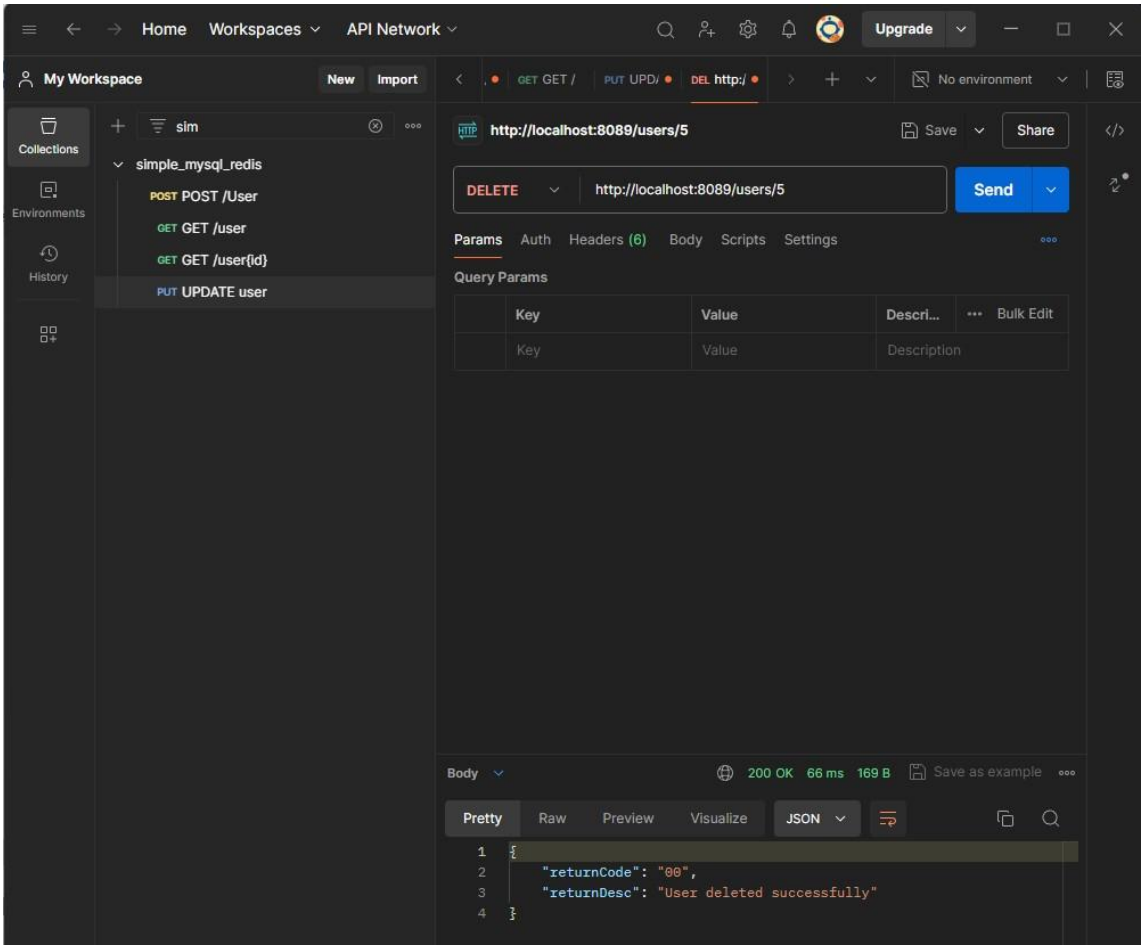
Selanjutnya pada bagian **Body** pilih **raw** dan ketikan json yang ingin diisikan baik id, nama, email, age yang mau diubah atau diperbaharui. Jika sudah selesai penginputkan lakukan send untuk lakukan Update data (Put). Jika berhasil maka muncul response JSON dari server.



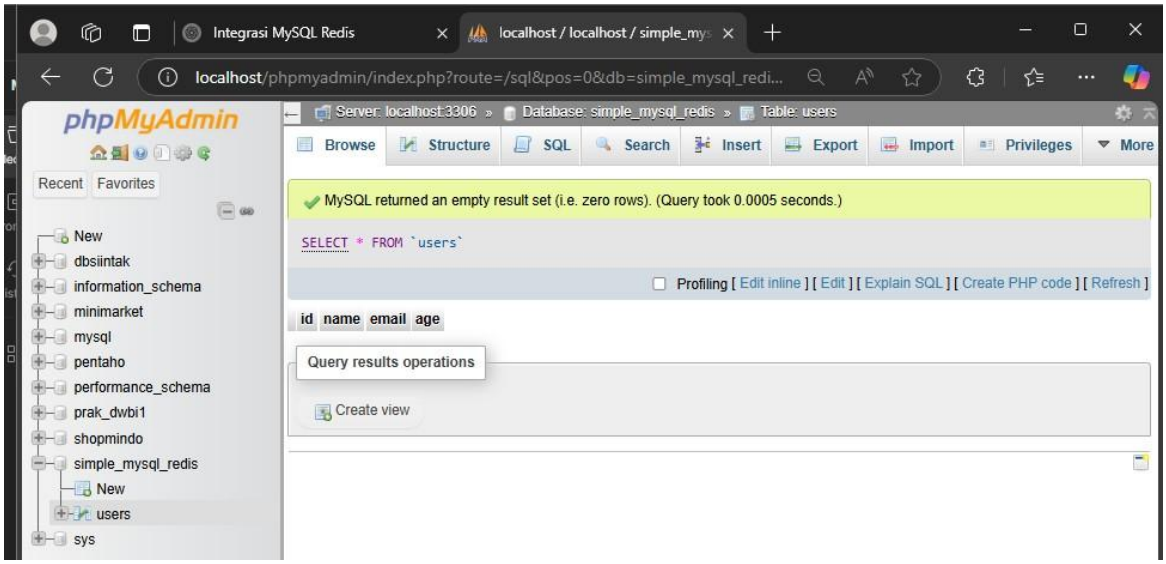
Data berhasil di localhost/database



12) Selanjutnya **Delete User**, lakukan **DELETE** pada Postman dengan url http://localhost:8089/users/1 lalu Send untuk menghapus salah satu data berdasarkan id. Jika berhasil maka akan muncul respon



Data berhasil di hapus dari tabel pada database /localhost



13) Semua proses API CRUD berhasil di lakukan dan success.